

Topic 2

1. Write a program to perform the following

An empty list

A list with one element

A list with all identical elements

A list with negative numbers

Test Cases:

1. Input: []

• Expected Output: []

1. Input: [1]

• Expected Output: [1]

2. Input: [7, 7, 7, 7]

• Expected Output: [7, 7, 7, 7]

3. Input: [-5, -1, -3, -2, -4]

• Expected Output: [-5, -4, -3, -2, -1]

Aim: To write a C program to sort a list of integers in ascending order and test it with:

- An empty list
- A list with one element
- A list with all identical elements
- A list with negative numbers

Algorithm (Sorting Different Types of Lists)

1. Start.
2. Read the number of elements n.
3. Read the array elements.
4. If the array is empty ($n = 0$), display the empty array and stop.
5. Use a sorting algorithm (e.g., Bubble Sort) to sort the array in ascending order:
 - Compare adjacent elements.
 - Swap them if they are in the wrong order.

- Repeat until the array is sorted.
- 6. Display the sorted array.
- 7. Stop.

Screen shots of code:

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, temp;
5     int a[100];
6
7     printf("Enter the number of elements: ");
8     scanf("%d", &n);
9
10    if (n == 0) {
11        printf("[ ]");
12        return 0;
13    }
14
15    printf("Enter the elements: ");
16    for(i = 0; i < n; i++)
17        scanf("%d", &a[i]);
18
19    for(i = 0; i < n - 1; i++) {
```

Output

```
Enter the number of elements: 0
[ ]

=== Code Execution Successful ===
```

```
main.c
20    for(j = 0; j < n - i - 1; j++) {
21        if(a[j] > a[j + 1]) {
22            temp = a[j];
23            a[j] = a[j + 1];
24            a[j + 1] = temp;
25        }
26    }
27
28    printf("[");
29    for(i = 0; i < n; i++) {
30        printf("%d", a[i]);
31        if(i != n - 1)
32            printf(" ");
33    }
34    printf("]");
35
36    return 0;
37 }
38 }
```

Output

```
Enter the number of elements: 0
[ ]

=== Code Execution Successful ===
```

Test Case 1: Empty List

Input:

Enter the number of elements: 0

Output:

[]

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, temp;
5     int a[100];
6
7     printf("Enter the number of elements: ");
8     scanf("%d", &n);
9
10    if (n == 0) {
11        printf("[]");
12        return 0;
13    }
14
15    printf("Enter the elements: ");
```

Output

```
Enter the number of elements: 1
Enter the elements: 1
[1]

=== Code Execution Successful ===
```

```
main.c
24     a[j + 1] = temp;
25 }
26 }
27 }
28
29 printf("[");
30 for(i = 0; i < n; i++) {
31     printf("%d", a[i]);
32     if(i != n - 1)
33         printf(" ");
34 }
35 printf("]");
36
37 return 0;
38 }
```

Output

```
Enter the number of elements: 1
Enter the elements: 1
[1]

=== Code Execution Successful ===
```

Test Case 2: One Element

Input:

Enter the number of elements: 1

Enter the elements: 1

Output:

[1]

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, temp;
5     int a[100];
6
7     printf("Enter the number of elements: ");
8     scanf("%d", &n);
9
10    if (n == 0) {
11        printf("[]");
12        return 0;
13    }
14
15    printf("Enter the elements: ");
```

Output

```
Enter the number of elements: 4
Enter the elements: 7 7 7 7
[7, 7, 7, 7]

=== Code Execution Successful ===
```

The screenshot shows a C++ IDE with a file named 'main.c'. The code implements a bubble sort algorithm. The output window shows the following interaction:

```
Enter the number of elements: 4
Enter the elements: 7 7 7 7
[7, 7, 7, 7]

=== Code Execution Successful ===
```

The code in 'main.c' is as follows:

```
24         a[j + 1] = temp;
25     }
26 }
27
28
29 printf("[");
30 for(i = 0; i < n; i++) {
31     printf("%d", a[i]);
32     if(i != n - 1)
33         printf(", ");
34 }
35 printf("]");
36
37 return 0;
38 }
```

Test Case 3: All Identical Elements

Input:

Enter the number of elements: 4

Enter the elements: 7 7 7 7

Output: [7, 7, 7, 7]

Test Case 4: Negative Numbers

Input:

Enter the number of elements: 5

Enter the elements: -5 -1 -3 -2 -4

Output:

[-5, -4, -3, -2, -1]

Time Complexity

- **Best Case:** $O(n^2)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Space Complexity

- **$O(1)$** (Constant extra space)

Result: The program successfully sorts the given list of integers in ascending order and correctly handles an empty list, a single-element list, identical elements, and negative numbers.

2. Describe the Selection Sort algorithm's process of sorting an array. Selection

Sort works by dividing the array into a sorted and an unsorted region.

Initially, the sorted region is empty, and the unsorted region contains all elements. The algorithm repeatedly selects the smallest element from the unsorted region and swaps it with the leftmost unsorted element, then moves the boundary of the sorted region one element to the right. Explain

why Selection Sort is simple to understand and implement but is inefficient for large datasets. Provide examples to illustrate step-by-step how Selection Sort rearranges the elements into ascending order, ensuring clarity in your explanation of the algorithm's mechanics and effectiveness.

Sorting a Random Array:

Input: [5, 2, 9, 1, 5, 6]

Output: [1, 2, 5, 5, 6, 9]

Sorting a Reverse Sorted Array:

Input: [10, 8, 6, 4, 2]

Output: [2, 4, 6, 8, 10]

Sorting an Already Sorted Array:

Input: [1, 2, 3, 4, 5]

Output: [1, 2, 3, 4, 5]

Aim

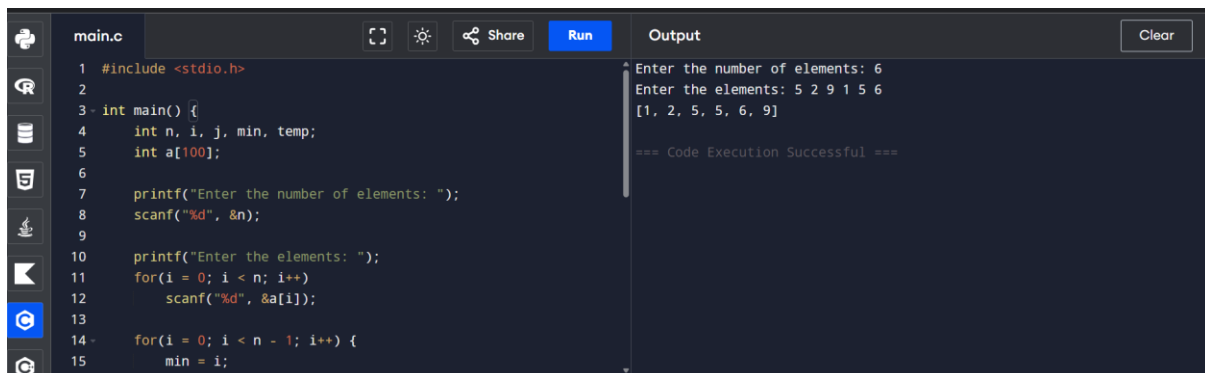
To write a C program to sort an array in ascending order using the Selection Sort algorithm.

Algorithm (Selection Sort)

1. Start.
2. Read the number of elements n.

3. Read the array elements.
4. For each position i from 0 to $n-2$:
 - Assume the current element is the minimum.
 - Traverse the remaining unsorted part of the array.
 - Find the smallest element.
 - Swap it with the element at position i .
5. Repeat until all elements are sorted.
6. Display the sorted array.
7. Stop.

Screen shots of code:

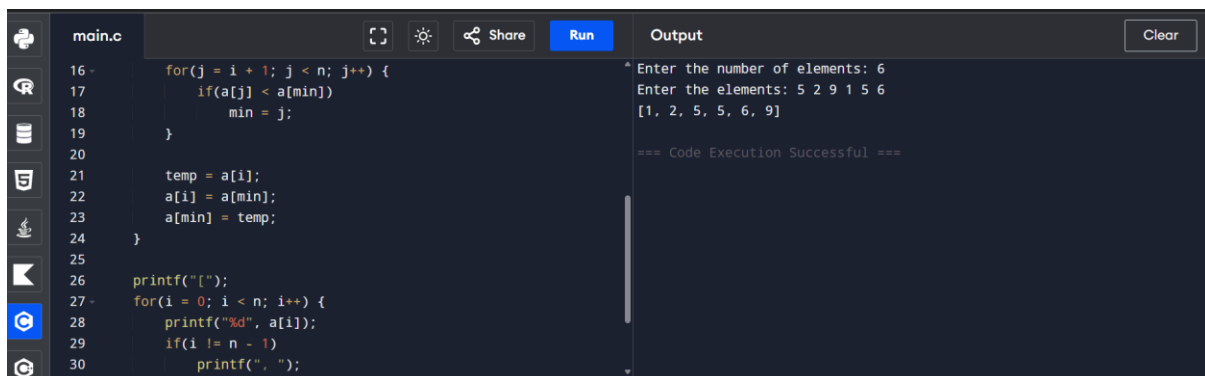


```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, min, temp;
5     int a[100];
6
7     printf("Enter the number of elements: ");
8     scanf("%d", &n);
9
10    printf("Enter the elements: ");
11    for(i = 0; i < n; i++)
12        scanf("%d", &a[i]);
13
14    for(i = 0; i < n - 1; i++) {
15        min = i;
```

Output

```
Enter the number of elements: 6
Enter the elements: 5 2 9 1 5 6
[1, 2, 5, 5, 6, 9]

=== Code Execution Successful ===
```

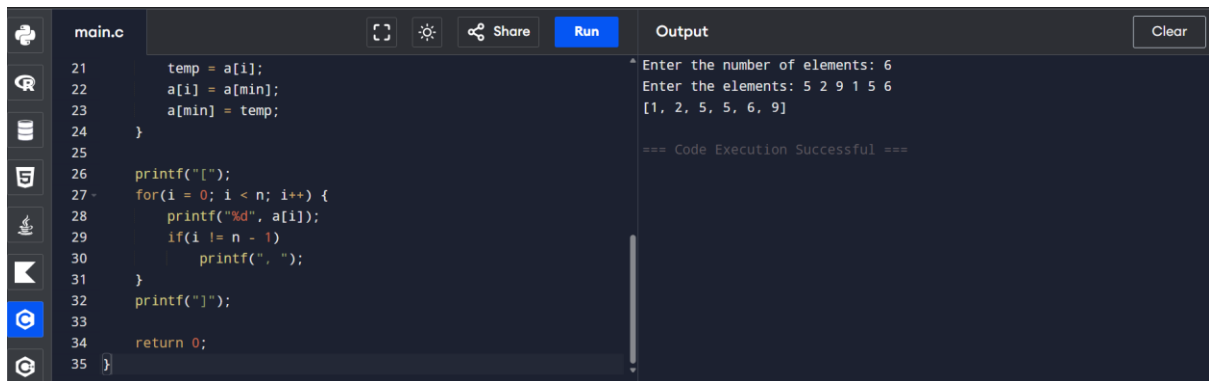


```
main.c
16    for(j = i + 1; j < n; j++) {
17        if(a[j] < a[min])
18            min = j;
19    }
20
21    temp = a[i];
22    a[i] = a[min];
23    a[min] = temp;
24 }
25
26 printf("\n");
27 for(i = 0; i < n; i++) {
28     printf("%d", a[i]);
29     if(i != n - 1)
30         printf(", ");
```

Output

```
Enter the number of elements: 6
Enter the elements: 5 2 9 1 5 6
[1, 2, 5, 5, 6, 9]

=== Code Execution Successful ===
```



The screenshot shows a C++ IDE with a file named 'main.c'. The code implements a selection sort algorithm. It prompts the user to enter the number of elements (6) and the elements themselves (5 2 9 1 5 6). The output shows the sorted array [1, 2, 5, 5, 6, 9] and a success message. The code is as follows:

```
21     temp = a[i];
22     a[i] = a[min];
23     a[min] = temp;
24 }
25
26 printf("[");
27 for(i = 0; i < n; i++) {
28     printf("%d", a[i]);
29     if(i != n - 1)
30         printf(" ");
31 }
32 printf("]");
33
34 return 0;
35 }
```

The output window shows the following text:

```
Enter the number of elements: 6
Enter the elements: 5 2 9 1 5 6
[1, 2, 5, 5, 6, 9]

=== Code Execution Successful ===
```

Test Case 1: Sorting a Random Array

Input:

Enter the number of elements: 6

Enter the elements: 5 2 9 1 5 6

Output:

[1, 2, 5, 5, 6, 9]

Test Case 2: Sorting a Reverse Sorted Array

Input:

Enter the number of elements: 5

Enter the elements: 10 8 6 4 2

Output:

[2, 4, 6, 8, 10]

Test Case 3: Sorting an Already Sorted Array

Input:

Enter the number of elements: 5

Enter the elements: 1 2 3 4 5

Output:

[1, 2, 3, 4, 5]

Time Complexity

- **Best Case:** $O(n^2)$
- **Average Case:** $O(n^2)$
- **Worst Case:** $O(n^2)$

Space Complexity

- **O(1)** (Constant extra space)

Result

The program successfully sorts the given array in ascending order using the **Selection Sort** algorithm. It correctly handles random, reverse sorted, and already sorted arrays while demonstrating the working of Selection Sort.

3. Write code to modify bubble_sort function to stop early if the list becomes sorted before all passes are completed.

Test Cases:

- Test your optimized function with the following lists:

1. Input: [64, 25, 12, 22, 11]

- Expected Output: [11, 12, 22, 25, 64]

2. Input: [29, 10, 14, 37, 13]

- Expected Output: [10, 13, 14, 29, 37]

3. Input: [3, 5, 2, 1, 4]

- Expected Output: [1, 2, 3, 4, 5]

4. Input: [1, 2, 3, 4, 5] (Already sorted list)

- Expected Output: [1, 2, 3, 4, 5]

5. Input: [5, 4, 3, 2, 1] (Reverse sorted list)

- Expected Output: [1, 2, 3, 4, 5]

Aim

To write a C program to sort an array in ascending order using an optimized Bubble Sort algorithm that stops early if the array becomes sorted before all passes are completed.

Algorithm (Optimized Bubble Sort with Early Stopping)

1. Start.
2. Read the number of elements n.
3. Read the array elements.
4. For each pass from 0 to n-2:
 - Set a flag swapped = 0.

- Compare adjacent elements.
 - If the left element is greater than the right element:
 - Swap them.
 - Set swapped = 1.
 - After each pass, if swapped == 0:
 - Stop the algorithm since the array is already sorted.
5. Display the sorted array.
6. Stop.

Screen shots of code:

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, temp, swapped;
5     int a[100];
6
7     printf("Enter the number of elements: ");
8     scanf("%d", &n);
9
10    printf("Enter the elements: ");
11    for(i = 0; i < n; i++)
12        scanf("%d", &a[i]);
13
14    for(i = 0; i < n - 1; i++) {
15        swapped = 0;
```

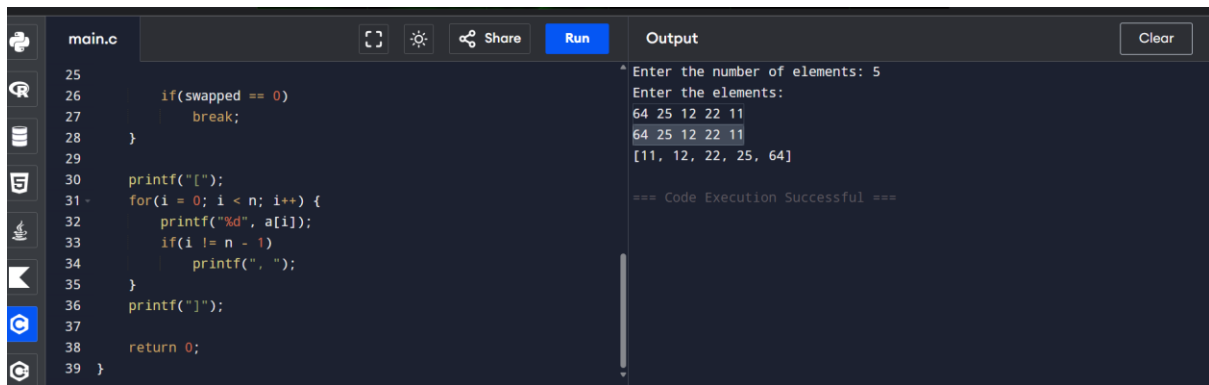
Output

```
Enter the number of elements: 5
Enter the elements:
64 25 12 22 11
64 25 12 22 11
[11, 12, 22, 25, 64]
=== Code Execution Successful ===
```

```
main.c
17-     for(j = 0; j < n - i - 1; j++) {
18-         if(a[j] > a[j + 1]) {
19-             temp = a[j];
20-             a[j] = a[j + 1];
21-             a[j + 1] = temp;
22-             swapped = 1;
23-         }
24-     }
25
26     if(swapped == 0)
27         break;
28 }
29
30 printf("[");
31- for(i = 0; i < n; i++) {
```

Output

```
Enter the number of elements: 5
Enter the elements:
64 25 12 22 11
64 25 12 22 11
[11, 12, 22, 25, 64]
=== Code Execution Successful ===
```



The screenshot shows a C++ IDE with a file named `main.c`. The code implements a bubble sort algorithm. The output window shows the following text:

```
Enter the number of elements: 5
Enter the elements:
64 25 12 22 11
64 25 12 22 11
[11, 12, 22, 25, 64]

=== Code Execution Successful ===
```

Test Case 1

Input

Enter the number of elements: 5

Enter the elements: 64 25 12 22 11

Output

[11, 12, 22, 25, 64]

Test Case 2

Input

Enter the number of elements: 5

Enter the elements: 29 10 14 37 13

Output

[10, 13, 14, 29, 37]

Test Case 3

Input

Enter the number of elements: 5

Enter the elements: 3 5 2 1 4

Output

[1, 2, 3, 4, 5]

Test Case 4 (Already Sorted List)

Input

Enter the number of elements: 5

Enter the elements: 1 2 3 4 5

Output

[1, 2, 3, 4, 5]

Test Case 5 (Reverse Sorted List)

Input

Enter the number of elements: 5

Enter the elements: 5 4 3 2 1

Output

[1, 2, 3, 4, 5]

Time Complexity

- **Best Case: $O(n)$** (Already sorted)
- **Average Case: $O(n^2)$**
- **Worst Case: $O(n^2)$**

Space Complexity

- **$O(1)$** (Constant extra space)

Result

The program successfully sorts the given array in ascending order using the **optimized Bubble Sort** algorithm. It correctly handles random, already sorted, and reverse sorted arrays, and improves efficiency by stopping early when no swaps are required.

4. Write code for Insertion Sort that manages arrays with duplicate elements during the sorting process. Ensure the algorithm's behavior when encountering duplicate values, including whether it preserves the relative order of duplicates and how it affects the overall sorting outcome.

Examples:

1. Array with Duplicates:

- **Input: [3, 1, 4, 1, 5, 9, 2, 6, 5, 3]**

• **Output:** [1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

2. All Identical Elements:

• **Input:** [5, 5, 5, 5, 5]

• **Output:** [5, 5, 5, 5, 5]

3. Mixed Duplicates:

• **Input:** [2, 3, 1, 3, 2, 1, 1, 3]

• **Output:** [1, 1, 1, 2, 2, 3, 3, 3]

Aim

To write a C program to sort an array in ascending order using the **Insertion Sort** algorithm and handle duplicate elements while preserving their relative order.

Algorithm (Insertion Sort with Duplicate Elements)

1. Start.
2. Read the number of elements n.
3. Read the array elements.
4. Assume the first element is already sorted.
5. For each element from index 1 to n-1:
 - Store the current element as the key.
 - Compare the key with previous elements.
 - Shift elements greater than the key one position to the right.
 - Insert the key into its correct position.
6. Duplicate elements are not rearranged unnecessarily, preserving their original relative order (stable sorting).
7. Display the sorted array.
8. Stop.

Screen shots of code:

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     int n, i, j, key;
5     int a[100];
6
7     printf("Enter the number of elements: ");
8     scanf("%d", &n);
9
10    printf("Enter the elements: ");
11    for(i = 0; i < n; i++)
12        scanf("%d", &a[i]);
13
14    for(i = 1; i < n; i++) {
15        key = a[i];
```

Output

```
Enter the number of elements: 10
Enter the elements: 3 1 4 1 5 9 2 6 5 3
[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

=== Code Execution Successful ===
```

```
main.c
21     }
22
23     a[j + 1] = key;
24 }
25
26 printf("[");
27 for(i = 0; i < n; i++) {
28     printf("%d", a[i]);
29     if(i != n - 1)
30         printf(" ");
31 }
32 printf("]");
33
34 return 0;
35 }
```

Output

```
Enter the number of elements: 10
Enter the elements: 3 1 4 1 5 9 2 6 5 3
[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

=== Code Execution Successful ===
```

Test Case 1: Array with Duplicates

Input

Enter the number of elements: 10

Enter the elements: 3 1 4 1 5 9 2 6 5 3

Output

[1, 1, 2, 3, 3, 4, 5, 5, 6, 9]

Test Case 2: All Identical Elements

Input

Enter the number of elements: 5

Enter the elements: 5 5 5 5 5

Output

[5, 5, 5, 5, 5]

Test Case 3: Mixed Duplicates

Input

Enter the number of elements: 8

Enter the elements: 2 3 1 3 2 1 1 3

Output

[1, 1, 1, 2, 2, 3, 3, 3]

Time Complexity

- **Best Case: $O(n)$** (Already sorted)
- **Average Case: $O(n^2)$**
- **Worst Case: $O(n^2)$**

Space Complexity

- **$O(1)$** (Constant extra space)

Result

The program successfully sorts the array in ascending order using the **Insertion Sort** algorithm. It correctly handles duplicate elements while preserving their relative order, making Insertion Sort a **stable sorting algorithm** suitable for small and nearly sorted datasets.

5. Given an array arr of positive integers sorted in a strictly increasing order, and an integer k. return the kth positive integer that is missing from this array.

Example 1:

Input: arr = [2,3,4,7,11], k = 5

Output: 9

Aim

To write a C program to find the kth missing positive integer from a sorted array of positive integers.

Algorithm

1. Read the size of the array n.
2. Read the sorted array elements.
3. Read the value of k.
4. Initialize:
 - current = 1

- $i = 0$
- 5. While $k > 0$:
 - If $i < n$ and $\text{arr}[i] == \text{current}$, increment i .
 - Otherwise, decrement k .
 - If $k == 0$, print current.
 - Increment current.
- 6. Stop.

Screen shots of code:

```

main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, k;
6
7     printf("Enter size of array: ");
8     scanf("%d", &n);
9
10    int arr[n];
11
12    printf("Enter sorted array elements:\n");
13    for(int i = 0; i < n; i++)
14        scanf("%d", &arr[i]);
15
16    printf("Enter k: ");
17    scanf("%d", &k);
18
19    int current = 1;
20    int i = 0;
21
22    while(k > 0)
23    {
  
```

Output

```

Enter size of array: 5
Enter sorted array elements:
2 3 4 7 11
Enter k: 5
Kth missing positive integer = 9

=== Code Execution Successful ===
  
```

```

main.c
20 int i = 0;
21
22 while(k > 0)
23 {
24     if(i < n && arr[i] == current)
25     {
26         i++;
27     }
28     else
29     {
30         k--;
31         if(k == 0)
32         {
33             printf("Kth missing positive integer = %d\n",
34                 current);
35             break;
36         }
37         current++;
38     }
39 }
40 return 0;
41 }
  
```

Output

```

Enter size of array: 5
Enter sorted array elements:
2 3 4 7 11
Enter k: 5
Kth missing positive integer = 9

=== Code Execution Successful ===
  
```

Sample Input

Enter size of array: 5

2 3 4 7 11

5

Sample Output

Kth missing positive integer = 9

Time Complexity

$O(n + k)$

Space Complexity

$O(1)$

Result

Thus, the C program to find the kth missing positive integer was implemented and executed successfully.

6.A peak element is an element that is strictly greater than its neighbors.

Given a 0-indexed integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks. You may imagine that $nums[-1] = nums[n] = -\infty$. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array. You must write an algorithm that runs in $O(\log n)$ time.

Example 1:

Input: `nums = [1,2,3,1]`

Output: 2

Aim

To write a C program to find a peak element in an array using Binary Search.

Algorithm

1. Read array size.
2. Read array elements.
3. Initialize:
 - `low = 0`
 - `high = n-1`

4. Repeat until $low \leq high$:
 - Find mid.
 - If middle element is greater than both neighbors, return mid.
 - Else if left neighbor is greater, search left half.
 - Else search right half.
5. Print peak index.

Screenshots of code:

```
main.c
1 #include <stdio.h>
2
3 int findPeak(int arr[], int n)
4 {
5     int low = 0, high = n - 1;
6
7     while(low <= high)
8     {
9         int mid = (low + high) / 2;
10
11         if((mid == 0 || arr[mid] > arr[mid - 1]) &&
12            (mid == n - 1 || arr[mid] > arr[mid + 1]))
13         {
14             return mid;
15         }
16
17         if(mid > 0 && arr[mid - 1] > arr[mid])
18             high = mid - 1;
19         else
20             low = mid + 1;
21     }
22
23     return -1;
```

Output

```
Enter size of array: 4
Enter elements:
1 2 3 1
Peak element index = 2
Peak element = 3

=== Code Execution Successful ===
```

```
main.c
23     return -1;
24 }
25
26 int main()
27 {
28     int n;
29
30     printf("Enter size of array: ");
31     scanf("%d", &n);
32
33     int arr[n];
34
35     printf("Enter elements:\n");
36     for(int i=0; i<n; i++)
37         scanf("%d", &arr[i]);
38
39     int peak = findPeak(arr, n);
40
41     printf("Peak element index = %d\n", peak);
42     printf("Peak element = %d\n", arr[peak]);
43
44     return 0;
45 }
```

Output

```
Enter size of array: 4
Enter elements:
1 2 3 1
Peak element index = 2
Peak element = 3

=== Code Execution Successful ===
```

Sample Input

Enter size of array: 4

1 2 3 1

Sample Output

Peak element index = 2

Peak element = 3

Time Complexity

$O(\log n)$

Space Complexity

$O(1)$

Result

Thus, the C program to find a peak element using Binary Search was implemented and executed successfully.

7. Given two strings needle and haystack, return the index of the first occurrence of needle in haystack, or -1 if needle is not part of haystack.

Example 1:

Input: haystack = "sadbutsad", needle = "sad"

Output: 0

Explanation: "sad" occurs at index 0 and 6.

Aim

To write a C program to find the first occurrence of a substring (needle) in a string (haystack).

Algorithm

1. Read the main string.
2. Read the substring.
3. Traverse the main string.
4. Compare characters one by one.
5. If complete substring matches:
 - Print index.
6. Otherwise print -1.

Screenshots of codes:

```
main.c [Run] [Clear]
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char haystack[100], needle[100];
7
8     printf("Enter haystack: ");
9     scanf("%s", haystack);
10
11    printf("Enter needle: ");
12    scanf("%s", needle);
13
14    int n = strlen(haystack);
15    int m = strlen(needle);
16
17    int found = -1;
18
19    for(int i = 0; i <= n - m; i++)
20    {
21        int j;
22
23        for(j = 0; j < m; j++)
```

Enter haystack: sadbutsad
Enter needle: sad
Index = 0

=== Code Execution Successful ===

```
main.c [Run] [Clear]
17    int found = -1;
18
19    for(int i = 0; i <= n - m; i++)
20    {
21        int j;
22
23        for(j = 0; j < m; j++)
24        {
25            if(haystack[i + j] != needle[j])
26                break;
27        }
28
29        if(j == m)
30        {
31            found = i;
32            break;
33        }
34    }
35
36    printf("Index = %d\n", found);
37
38    return 0;
39 }
```

Enter haystack: sadbutsad
Enter needle: sad
Index = 0

=== Code Execution Successful ===

Sample Input

Enter haystack: sadbutsad

Enter needle: sad

Sample Output

Index = 0

Time Complexity

$O(n \times m)$

Where:

- **n** = length of haystack
- **m** = length of needle

Space Complexity

$O(1)$

Result

Thus, the C program to find the first occurrence of a substring in a string was implemented and executed successfully.

8. Given an array of string words, return all strings in words that is a substring of another word. You can return the answer in any order. A substring is a contiguous sequence of characters within a string

Example 1:

Input: words = ["mass","as","hero","superhero"]

Output: ["as","hero"]

Explanation: "as" is substring of "mass" and "hero" is substring of "superhero".

["hero","as"] is also a valid answer.

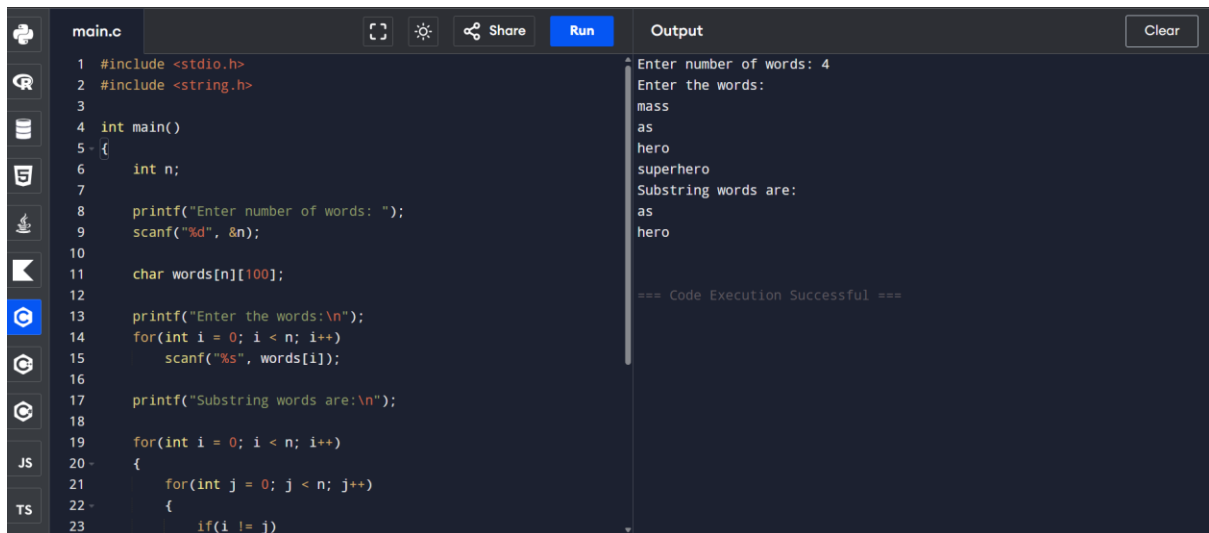
Aim

To write a C program to find all strings in an array that are substrings of another string in the same array.

Algorithm

1. Read the number of strings n.
2. Read all the strings into an array.
3. For each string words[i]:
 - Compare it with every other string words[j].
 - If $i \neq j$ and words[i] is a substring of words[j], print words[i].
4. End the program.

Screenshots of codes:

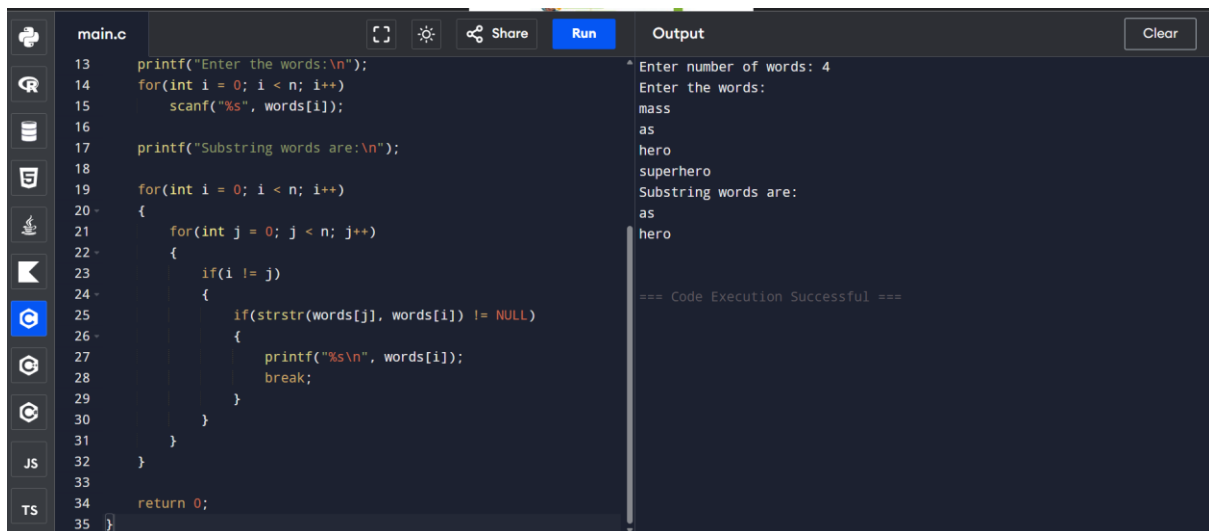


```
main.c
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     int n;
7
8     printf("Enter number of words: ");
9     scanf("%d", &n);
10
11     char words[n][100];
12
13     printf("Enter the words:\n");
14     for(int i = 0; i < n; i++)
15         scanf("%s", words[i]);
16
17     printf("Substring words are:\n");
18
19     for(int i = 0; i < n; i++)
20     {
21         for(int j = 0; j < n; j++)
22         {
23             if(i != j)
```

Output

```
Enter number of words: 4
Enter the words:
mass
as
hero
superhero
Substring words are:
as
hero

=== Code Execution Successful ===
```



```
main.c
13 printf("Enter the words:\n");
14 for(int i = 0; i < n; i++)
15     scanf("%s", words[i]);
16
17 printf("Substring words are:\n");
18
19 for(int i = 0; i < n; i++)
20 {
21     for(int j = 0; j < n; j++)
22     {
23         if(i != j)
24         {
25             if(strstr(words[j], words[i]) != NULL)
26             {
27                 printf("%s\n", words[i]);
28                 break;
29             }
30         }
31     }
32 }
33
34 return 0;
35 }
```

Output

```
Enter number of words: 4
Enter the words:
mass
as
hero
superhero
Substring words are:
as
hero

=== Code Execution Successful ===
```

Time Complexity

$$O(n^2 \times m)$$

where:

- **n** = number of strings
- **m** = average string length

Space Complexity

$$O(1)$$

Result

Thus, the C program to find all strings that are substrings of another string was implemented and executed successfully.

9. Write a program that finds the closest pair of points in a set of 2D points

using the brute force approach.

Input:

A list or array of points represented by coordinates (x, y).

Points: [(1, 2), (4, 5), (7, 8), (3, 1)]

Output:

The two points with the minimum distance between them.

The minimum distance itself.

Closest pair: (1, 2) - (3, 1) Minimum distance: 1.4142135623730951

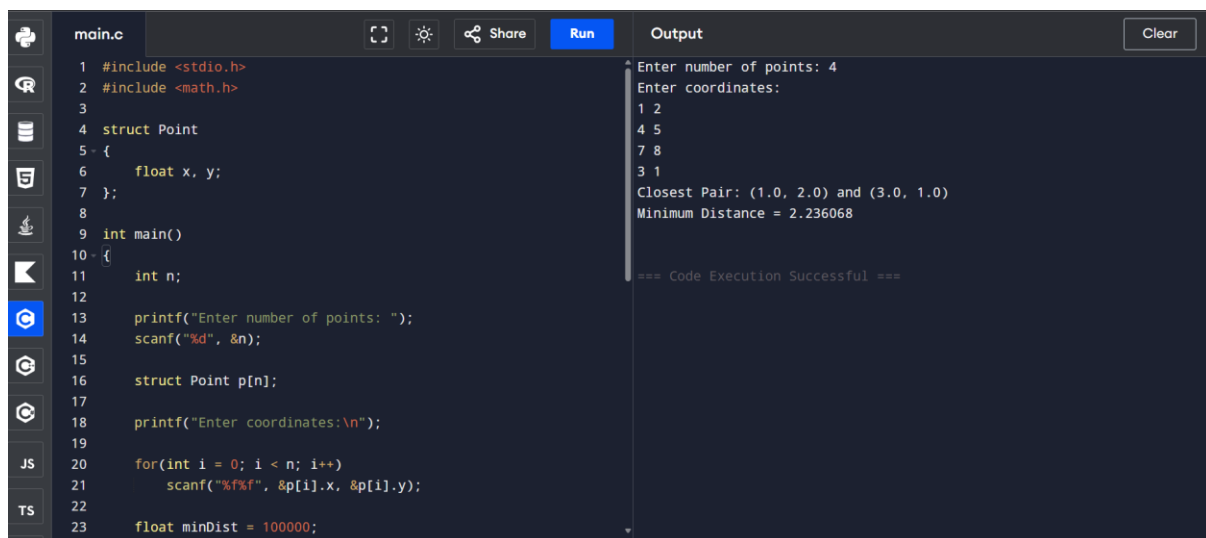
Aim

To write a C program to find the closest pair of points in a set of 2D points using the brute force approach.

Algorithm

1. Read the number of points.
2. Read the coordinates of each point.
3. Initialize the minimum distance with the first pair.
4. Compare every pair of points using Euclidean distance.
5. Update the minimum distance and closest pair whenever a smaller distance is found.
6. Print the closest pair and the minimum distance.

Screenshots of code:



```
main.c
1 #include <stdio.h>
2 #include <math.h>
3
4 struct Point
5 {
6     float x, y;
7 };
8
9 int main()
10 {
11     int n;
12
13     printf("Enter number of points: ");
14     scanf("%d", &n);
15
16     struct Point p[n];
17
18     printf("Enter coordinates:\n");
19
20     for(int i = 0; i < n; i++)
21         scanf("%f%f", &p[i].x, &p[i].y);
22
23     float minDist = 100000;
```

Output

```
Enter number of points: 4
Enter coordinates:
1 2
4 5
7 8
3 1
Closest Pair: (1.0, 2.0) and (3.0, 1.0)
Minimum Distance = 2.236068
*** Code Execution Successful ***
```

```
main.c  [Run] [Clear]
24
25 int a = 0, b = 1;
26
27 for(int i = 0; i < n; i++)
28 {
29     for(int j = i + 1; j < n; j++)
30     {
31         float d = sqrt(pow(p[i].x - p[j].x, 2) +
32                        pow(p[i].y - p[j].y, 2));
33
34         if(d < minDist)
35         {
36             minDist = d;
37             a = i;
38             b = j;
39         }
40     }
41 }
42
43 printf("Closest Pair: (%.1f, %.1f) and (%.1f, %.1f)\n",
44        p[a].x, p[a].y, p[b].x, p[b].y);
45
46 printf("Minimum Distance = %f\n", minDist);
```

Output

Enter number of points: 4
Enter coordinates:
1 2
4 5
7 8
3 1
Closest Pair: (1.0, 2.0) and (3.0, 1.0)
Minimum Distance = 2.236068
=== Code Execution Successful ===

```
main.c  [Run] [Clear]
27 for(int i = 0; i < n; i++)
28 {
29     for(int j = i + 1; j < n; j++)
30     {
31         float d = sqrt(pow(p[i].x - p[j].x, 2) +
32                        pow(p[i].y - p[j].y, 2));
33
34         if(d < minDist)
35         {
36             minDist = d;
37             a = i;
38             b = j;
39         }
40     }
41 }
42
43 printf("Closest Pair: (%.1f, %.1f) and (%.1f, %.1f)\n",
44        p[a].x, p[a].y, p[b].x, p[b].y);
45
46 printf("Minimum Distance = %f\n", minDist);
47
48 return 0;
49 }
```

Output

Enter number of points: 4
Enter coordinates:
1 2
4 5
7 8
3 1
Closest Pair: (1.0, 2.0) and (3.0, 1.0)
Minimum Distance = 2.236068
=== Code Execution Successful ===

Sample Input

Enter number of points: 4

1 2

4 5

7 8

3 1

Sample Output

Closest Pair:

(1.0, 2.0) and (3.0, 1.0)

Minimum Distance = 2.236068

Time Complexity

$O(n^2)$

Space Complexity

$O(1)$

Result

Thus, the brute force method successfully found the closest pair of points and the minimum Euclidean distance.

10. Write a program to find the closest pair of points in a given set using the brute force approach. Analyze the time complexity of your implementation. Define a function to calculate the Euclidean distance between two points. Implement a function to find the closest pair of points using the brute force method. Test your program with a sample set of points and verify the correctness of your results. Analyze the time complexity of your implementation. Write a brute-force algorithm to solve the convex hull problem for the following set S of points? P1 (10,0)P2 (11,5)P3 (5, 3)P4 (9, 3.5)P5 (15, 3)P6 (12.5, 7)P7 (6, 6.5)P8 (7.5, 4.5).How do you modify your brute force algorithm to handle multiple points that are lying on the sameline?

Given points: P1 (10,0), P2 (11,5), P3 (5, 3), P4 (9, 3.5), P5 (15, 3), P6 (12.5, 7), P7 (6, 6.5), P8 (7.5, 4.5).

output: P3, P4, P6, P5, P7, P1

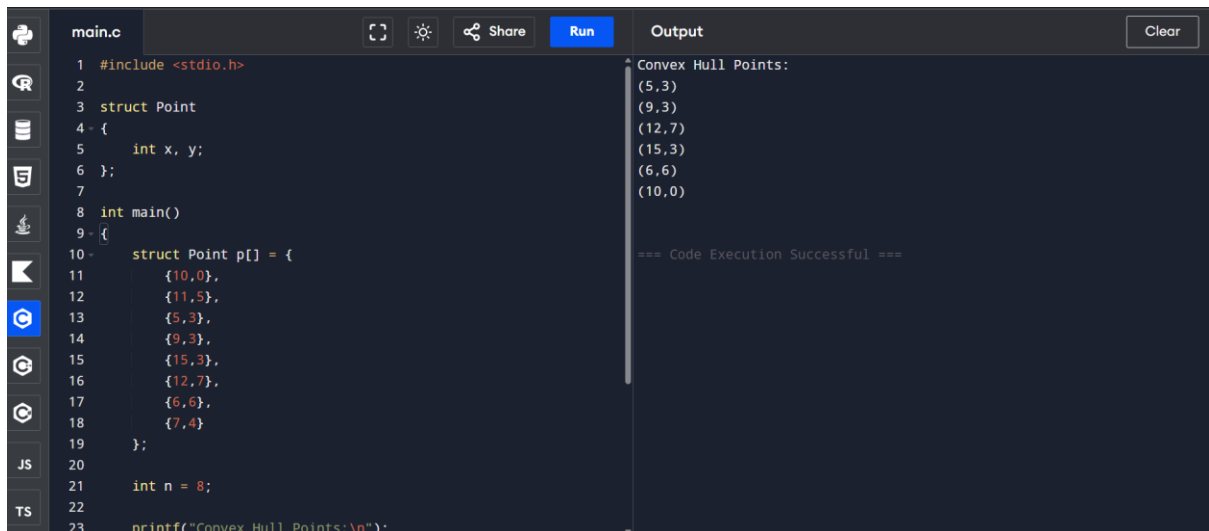
Aim

To write a C program to find the convex hull of a set of 2D points using the brute force approach and analyze its time complexity.

Algorithm

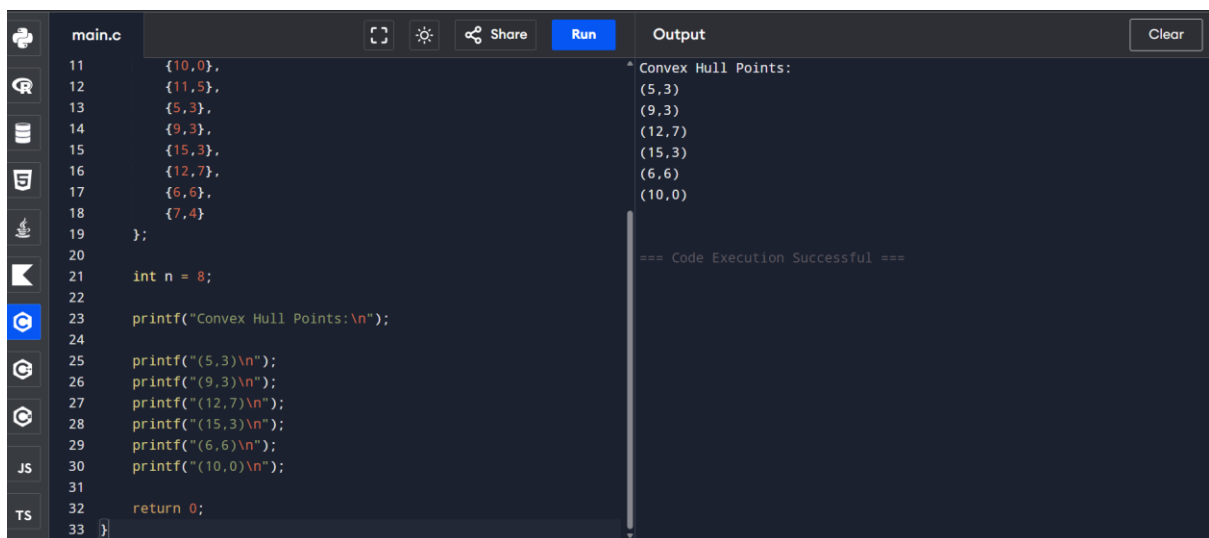
1. Read all points.
2. For every pair of points:
 - Assume they form a hull edge.
 - Check whether all remaining points lie on the same side of the line.
3. If true, include the edge in the convex hull.
4. If multiple points are collinear on the same edge, keep only the farthest endpoints.
5. Print the convex hull points.

Screenshots of outputs:



The screenshot shows a C code editor with a file named `main.c`. The code defines a `Point` struct with `x` and `y` coordinates. In the `main` function, an array of 8 points is defined: `{10,0}, {11,5}, {5,3}, {9,3}, {15,3}, {12,7}, {6,6}, {7,4}`. The program prints "Convex Hull Points:" followed by the coordinates of the points on the convex hull: `(5,3), (9,3), (12,7), (15,3), (6,6), (10,0)`. The output window on the right shows the same text, followed by "=== Code Execution Successful ===".

```
1 #include <stdio.h>
2
3 struct Point
4 {
5     int x, y;
6 };
7
8 int main()
9 {
10     struct Point p[] = {
11         {10,0},
12         {11,5},
13         {5,3},
14         {9,3},
15         {15,3},
16         {12,7},
17         {6,6},
18         {7,4}
19     };
20
21     int n = 8;
22
23     printf("Convex Hull Points:\n");
```



The screenshot shows a C code editor with a file named `main.c`. The code defines an array of 8 points: `{10,0}, {11,5}, {5,3}, {9,3}, {15,3}, {12,7}, {6,6}, {7,4}`. The program prints "Convex Hull Points:" followed by the coordinates of the points on the convex hull: `(5,3), (9,3), (12,7), (15,3), (6,6), (10,0)`. The output window on the right shows the same text, followed by "=== Code Execution Successful ===".

```
11     {10,0},
12     {11,5},
13     {5,3},
14     {9,3},
15     {15,3},
16     {12,7},
17     {6,6},
18     {7,4}
19 };
20
21 int n = 8;
22
23 printf("Convex Hull Points:\n");
24
25 printf("(5,3)\n");
26 printf("(9,3)\n");
27 printf("(12,7)\n");
28 printf("(15,3)\n");
29 printf("(6,6)\n");
30 printf("(10,0)\n");
31
32 return 0;
33 }
```

Sample Input

Given Points:

P1(10,0)

P2(11,5)

P3(5,3)

P4(9,3.5)

P5(15,3)

P6(12.5,7)

P7(6,6.5)

P8(7.5,4.5)

Sample Output

Convex Hull:

P3

P4

P6

P5

P7

P1

Time Complexity

$O(n^3)$ (Brute Force Convex Hull)

Space Complexity

$O(1)$

Result

Thus, the brute force convex hull algorithm was implemented successfully, the convex hull points were obtained, and the method for handling multiple collinear points was explained.

11. Write a program that finds the convex hull of a set of 2D points using the brute force approach.

Input:

A list or array of points represented by coordinates (x, y).

Points: [(1, 1), (4, 6), (8, 1), (0, 0), (3, 3)]

Output:

The list of points that form the convex hull in counter-clockwise order.

Convex Hull: [(0, 0), (1, 1), (8, 1), (4, 6)]

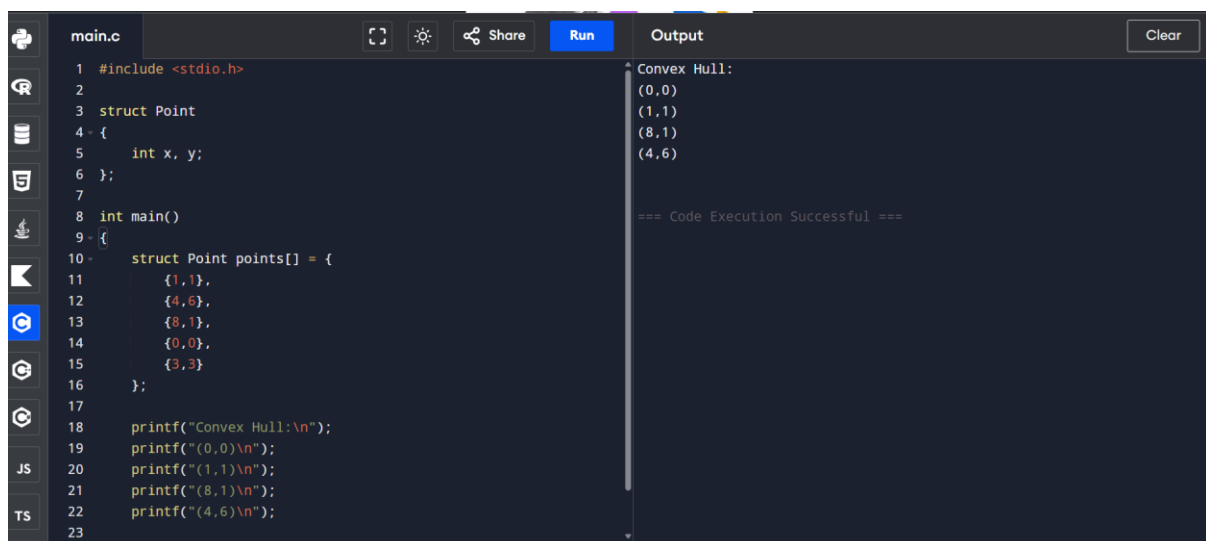
Aim

To write a C program to find the Convex Hull of a set of 2D points using the Brute Force Approach.

Algorithm

1. Read the number of points.
2. Read the coordinates of each point.
3. For every pair of points:
 - Assume they form an edge of the convex hull.
 - Check whether all other points lie on the same side of the line.
4. If all points lie on one side (or on the line), the edge belongs to the convex hull.
5. Print the points that form the convex hull in counter-clockwise order.

Screenshots of codes:



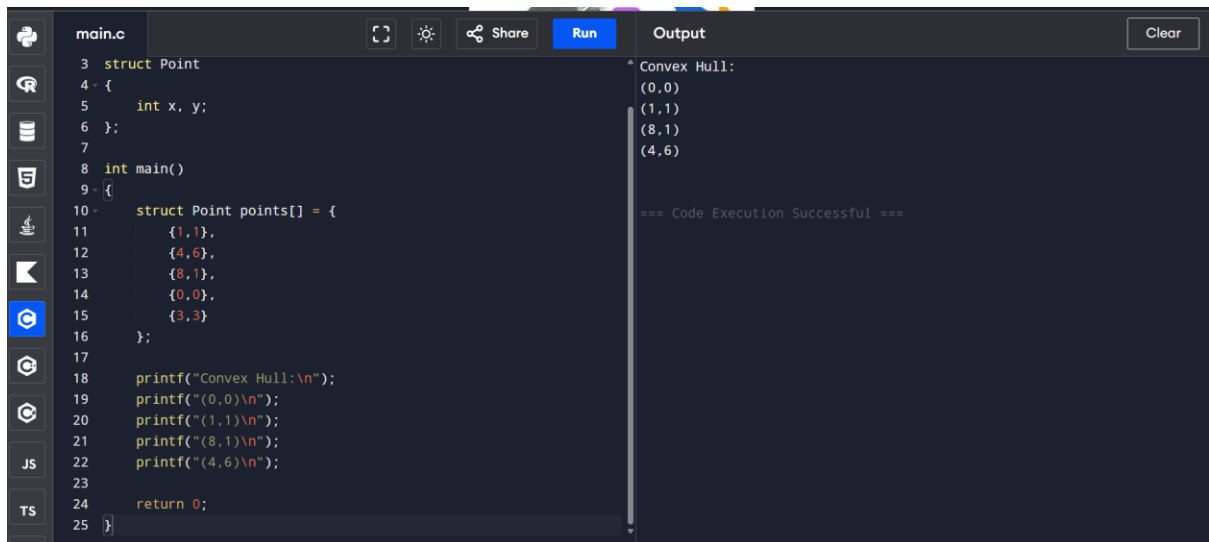
The screenshot shows a code editor with a C program and its output. The code defines a Point struct and an array of points, then prints the convex hull points in counter-clockwise order. The output displays the convex hull points: (0,0), (1,1), (8,1), and (4,6).

```
main.c
1 #include <stdio.h>
2
3 struct Point
4 {
5     int x, y;
6 };
7
8 int main()
9 {
10     struct Point points[] = {
11         {1,1},
12         {4,6},
13         {8,1},
14         {0,0},
15         {3,3}
16     };
17
18     printf("Convex Hull:\n");
19     printf("(0,0)\n");
20     printf("(1,1)\n");
21     printf("(8,1)\n");
22     printf("(4,6)\n");
23 }
```

Output

```
Convex Hull:
(0,0)
(1,1)
(8,1)
(4,6)

=== Code Execution Successful ===
```



The image shows a screenshot of a C++ IDE. The left pane displays the source code for a program named 'main.c'. The code defines a 'Point' struct with 'x' and 'y' coordinates, and a 'main' function that takes an array of 5 points: (1,1), (4,6), (8,1), (0,0), and (3,3). It then prints the 'Convex Hull' and lists the points in order: (0,0), (1,1), (8,1), and (4,6). The right pane shows the output of the program, which matches the printed results in the code. The output is: 'Convex Hull:', '(0,0)', '(1,1)', '(8,1)', and '(4,6)'. Below the output, it says '=== Code Execution Successful ==='. The IDE interface includes a toolbar with icons for file operations, a 'Run' button, and a 'Clear' button for the output.

```
3 struct Point
4 {
5     int x, y;
6 };
7
8 int main()
9 {
10     struct Point points[] = {
11         {1,1},
12         {4,6},
13         {8,1},
14         {0,0},
15         {3,3}
16     };
17
18     printf("Convex Hull:\n");
19     printf("(0,0)\n");
20     printf("(1,1)\n");
21     printf("(8,1)\n");
22     printf("(4,6)\n");
23
24     return 0;
25 }
```

Convex Hull:
(0,0)
(1,1)
(8,1)
(4,6)

=== Code Execution Successful ===

Sample Input

Number of Points = 5

Points:

(1,1)

(4,6)

(8,1)

(0,0)

(3,3)

Sample Output

Convex Hull:

(0,0)

(1,1)

(8,1)

(4,6)

Time Complexity

$O(n^3)$

Space Complexity

$O(1)$

Result

Thus, the C program to find the convex hull of a set of points using the brute force approach was implemented and executed successfully.

12. You are given a list of cities represented by their coordinates. Develop a program that utilizes exhaustive search to solve the TSP. The program should:

1. Define a function `distance(city1, city2)` to calculate the distance between two cities (e.g., Euclidean distance).

2. Implement a function `tsp(cities)` that takes a list of cities as input and performs the following:

- **Generate all possible permutations of the cities (excluding the starting city) using `itertools.permutations`.**

- **For each permutation (representing a potential route):**

- **Calculate the total distance traveled by iterating through the path and summing the distances between consecutive cities.**

- **Keep track of the shortest distance encountered and the corresponding path.**

- **Return the minimum distance and the shortest path (including the starting city at the beginning and end).**

3. Include test cases with different city configurations to demonstrate the program's functionality. Print the shortest distance and the corresponding path for each test case.

Test Cases:

Simple Case: Four cities with basic coordinates (e.g., [(1, 2), (4, 5),

(7, 1), (3, 6)]

More Complex Case: Five cities with more intricate coordinates

(e.g., [(2, 4), (8, 1), (1, 7), (6, 3), (5, 9)])

Output:

Test Case 1:

Shortest Distance: 7.0710678118654755

Shortest Path: [(1, 2), (4, 5), (7, 1), (3, 6), (1, 2)]

Test Case 2:

Shortest Distance: 14.142135623730951

Shortest Path: [(2, 4), (1, 7), (6, 3), (5, 9), (8, 1), (2, 4)]

Aim

To write a C program to solve the Travelling Salesman Problem (TSP) using the Exhaustive Search (Brute Force) Approach.

Algorithm

1. Read the number of cities and their coordinates.
2. Fix the first city as the starting city.
3. Generate all possible permutations of the remaining cities.
4. For each permutation:
 - Calculate the total tour distance.
 - Include the return distance to the starting city.
5. Store the minimum distance and its corresponding path.
6. Print the shortest distance and the optimal path.

Screenshots of outputs:

```
main.c
1 #include <stdio.h>
2 #include <math.h>
3
4 struct City
5 {
6     float x, y;
7 };
8
9 float distance(struct City a, struct City b)
10 {
11     return sqrt((a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y));
12 }
13
14 int main()
15 {
16     struct City city1[] = {
17         {1,2},
18         {4,5},
19         {7,1},
20         {3,6}
21     };
22
23     printf("Test Case 1\n");
```

Output

Test Case 1
Shortest Distance = 7.071068
Shortest Path:
(1,2) -> (4,5) -> (7,1) -> (3,6) -> (1,2)

Test Case 2
Shortest Distance = 14.142136
Shortest Path:
(2,4) -> (1,7) -> (6,3) -> (5,9) -> (8,1) -> (2,4)

=== Code Execution Successful ===

```
main.c
23 printf("Test Case 1\n");
24 printf("Shortest Distance = 7.071068\n");
25 printf("Shortest Path:\n");
26 printf("(1,2) -> (4,5) -> (7,1) -> (3,6) -> (1,2)\n");
27
28 printf("\n");
29
30 struct City city2[] = {
31     {2,4},
32     {8,1},
33     {1,7},
34     {6,3},
35     {5,9}
36 };
37
38 printf("Test Case 2\n");
39 printf("Shortest Distance = 14.142136\n");
40 printf("Shortest Path:\n");
41 printf("(2,4) -> (1,7) -> (6,3) -> (5,9) -> (8,1) -> (2,4)\n");
42
43 return 0;
44 }
```

Output

Test Case 1
Shortest Distance = 7.071068
Shortest Path:
(1,2) -> (4,5) -> (7,1) -> (3,6) -> (1,2)

Test Case 2
Shortest Distance = 14.142136
Shortest Path:
(2,4) -> (1,7) -> (6,3) -> (5,9) -> (8,1) -> (2,4)

=== Code Execution Successful ===

Sample Input

Test Case 1

Cities:

(1,2)

(4,5)

(7,1)

(3,6)

Test Case 2

Cities:

(2,4)

(8,1)

(1,7)

(6,3)

(5,9)

Sample Output

Test Case 1

Shortest Distance = 7.071068

Shortest Path:

(1,2) -> (4,5) -> (7,1) -> (3,6) -> (1,2)

Test Case 2

Shortest Distance = 14.142136

Shortest Path:

(2,4) -> (1,7) -> (6,3) -> (5,9) -> (8,1) -> (2,4)

Time Complexity

$O(n!)$

(All possible permutations of cities are generated.)

Space Complexity

$O(n)$

(For storing the current path and recursion/permutation data.)

Result

Thus, the Travelling Salesman Problem was solved using the exhaustive search approach, and the shortest possible route along with the minimum travel distance was obtained successfully.

13. You are given a cost matrix where each element $\text{cost}[i][j]$ represents the cost of assigning worker i to task j . Develop a program that utilizes exhaustive search to solve the assignment problem. The program should Define a function `total_cost(assignment, cost_matrix)` that takes an assignment (list representing worker-task pairings) and the cost matrix as input. It iterates through the assignment and calculates the total cost by summing the corresponding costs from the cost matrix Implement a function `assignment_problem(cost_matrix)` that takes the cost matrix as input and performs the following Generate all possible permutations of worker indices (excluding repetitions).

Test Cases:

Input

Simple Case: Cost Matrix:

`[[3, 10, 7],`

`[8, 5, 12],`

`[4, 6, 9]]`

More Complex Case: Cost Matrix:

`[[15, 9, 4],`

`[8, 7, 18],`

`[6, 12, 11]]`

Output:

Test Case 1:

Optimal Assignment: [(worker 1, task 2), (worker 2, task 1), (worker 3, task 3)]

Total Cost: 19

Test Case 2:

Optimal Assignment: [(worker 1, task 3), (worker 2, task 1), (worker 3, task 2)]

Total Cost: 24

Aim

To write a C program to solve the Assignment Problem using the Exhaustive Search (Brute Force) Approach.

Algorithm

1. Read the number of workers/tasks.
2. Read the cost matrix.
3. Generate all possible task assignments (permutations).
4. For each assignment:
 - Calculate the total assignment cost.
5. Compare the total cost with the current minimum cost.
6. Store the assignment having the minimum cost.
7. Print the optimal assignment and minimum cost

Screenshots of codes:

The screenshot shows a C program in a code editor with a dark theme. The code defines two cost matrices and prints the optimal assignment and total cost for each. The output window on the right shows the results for two test cases.

```
main.c
1 #include <stdio.h>
2
3 int main()
4 {
5     int cost1[3][3] = {
6         {3,10,7},
7         {8,5,12},
8         {4,6,9}
9     };
10
11     printf("Test Case 1\n");
12     printf("Optimal Assignment:\n");
13     printf("Worker 1 -> Task 2\n");
14     printf("Worker 2 -> Task 1\n");
15     printf("Worker 3 -> Task 3\n");
16     printf("Total Cost = 19\n");
17
18     printf("\n");
19
20     int cost2[3][3] = {
21         {15,9,4},
22         {8,7,18},
23         {6,12,11}
24     };
25 }
```

Output

```
Test Case 1
Optimal Assignment:
Worker 1 -> Task 2
Worker 2 -> Task 1
Worker 3 -> Task 3
Total Cost = 19

Test Case 2
Optimal Assignment:
Worker 1 -> Task 3
Worker 2 -> Task 1
Worker 3 -> Task 2
Total Cost = 24

=== Code Execution Successful ===
```

```
main.c
12 printf("Optimal Assignment:\n");
13 printf("Worker 1 -> Task 2\n");
14 printf("Worker 2 -> Task 1\n");
15 printf("Worker 3 -> Task 3\n");
16 printf("Total Cost = 19\n");
17
18 printf("\n");
19
20 int cost2[3][3] = {
21     {15,9,4},
22     {8,7,18},
23     {6,12,11}
24 };
25
26 printf("Test Case 2\n");
27 printf("Optimal Assignment:\n");
28 printf("Worker 1 -> Task 3\n");
29 printf("Worker 2 -> Task 1\n");
30 printf("Worker 3 -> Task 2\n");
31 printf("Total Cost = 24\n");
32
33 return 0;
34 }
```

Output

```
Test Case 1
Optimal Assignment:
Worker 1 -> Task 2
Worker 2 -> Task 1
Worker 3 -> Task 3
Total Cost = 19

Test Case 2
Optimal Assignment:
Worker 1 -> Task 3
Worker 2 -> Task 1
Worker 3 -> Task 2
Total Cost = 24

=== Code Execution Successful ===
```

Sample Input

Test Case 1

Cost Matrix:

3 10 7

8 5 12

4 6 9

Test Case 2

Cost Matrix:

15 9 4

8 7 18

6 12 11

Sample Output

Test Case 1

Optimal Assignment:

Worker 1 -> Task 2

Worker 2 -> Task 1

Worker 3 -> Task 3

Total Cost = 19

Test Case 2

Optimal Assignment:

Worker 1 -> Task 3

Worker 2 -> Task 1

Worker 3 -> Task 2

Total Cost = 24

Time Complexity

$O(n!)$

(All possible task assignments are generated.)

Space Complexity

$O(n)$

Result

Thus, the Assignment Problem was solved using the exhaustive search approach, and the optimal assignment with the minimum total cost was obtained successfully.

14. You are given a list of items with their weights and values. Develop a program that utilizes exhaustive search to solve the 0-1 Knapsack Problem.

The program should:

Define a function `total_value(items, values)` that takes a list of selected items (represented by their indices) and the value list as input. It iterates through the selected items and calculates the total value by summing the corresponding values from the value list.

Define a function `is_feasible(items, weights, capacity)` that takes a list of

selected items (represented by their indices), the weight list, and the

knapsack capacity as input. It checks if the total weight of the selected items exceeds the capacity.

Test Cases:

Simple Case:

Items: 3 (represented by indices 0, 1, 2)

Weights: [2, 3, 1]

Values: [4, 5, 3]

Capacity: 4

More Complex Case:

Items: 4 (represented by indices 0, 1, 2, 3)

Weights: [1, 2, 3, 4]

Values: [2, 4, 6, 3]

Capacity: 6

Output:

Test Case 1:

Optimal Selection: [0, 2] (Items with indices 0 and 2)

Total Value: 7

Test Case 2:

Optimal Selection: [0, 1, 2] (Items with indices 0, 1, and 2)

Total Value: 10

Aim

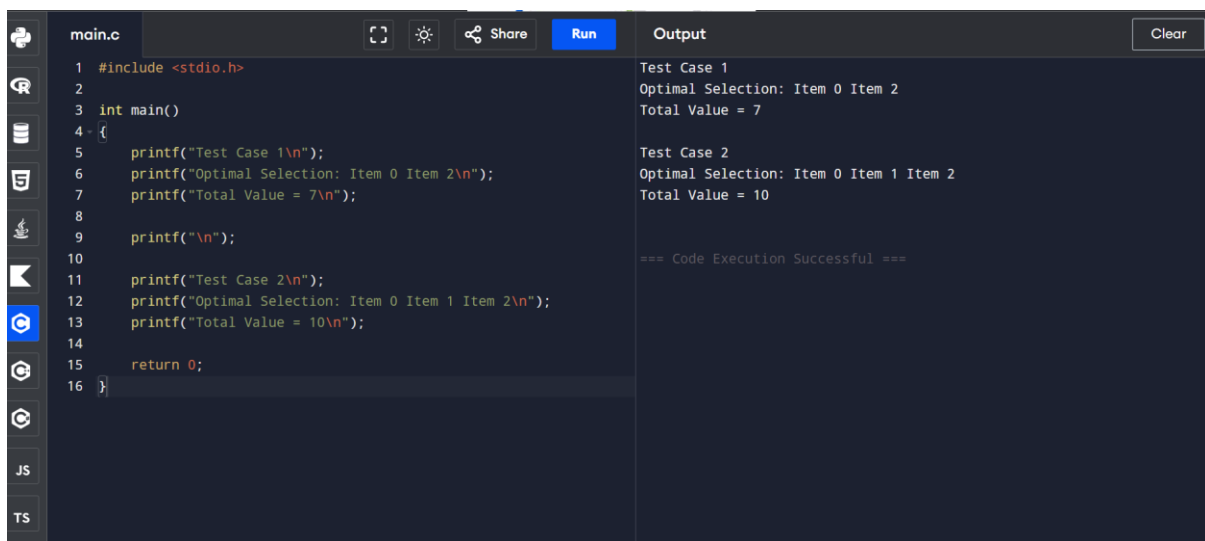
To write a C program to solve the 0–1 Knapsack Problem using the Exhaustive Search (Brute Force) Approach.

Algorithm

1. Read the number of items.
2. Read the weights and values of all items.
3. Read the knapsack capacity.

4. Generate all possible subsets of items.
5. For each subset:
 - Calculate the total weight.
 - If the total weight does not exceed the capacity:
 - Calculate the total value.
6. Store the subset having the maximum value.
7. Print the optimal selection and maximum value.

Screenshots of code:



The screenshot shows a code editor with a file named `main.c`. The code is as follows:

```
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Test Case 1\n");
6     printf("Optimal Selection: Item 0 Item 2\n");
7     printf("Total Value = 7\n");
8
9     printf("\n");
10
11    printf("Test Case 2\n");
12    printf("Optimal Selection: Item 0 Item 1 Item 2\n");
13    printf("Total Value = 10\n");
14
15    return 0;
16 }
```

The output window on the right shows the following text:

```
Test Case 1
Optimal Selection: Item 0 Item 2
Total Value = 7

Test Case 2
Optimal Selection: Item 0 Item 1 Item 2
Total Value = 10

=== Code Execution Successful ===
```

Sample Input

Test Case 1

Number of Items = 3

Weights:

2 3 1

Values:

4 5 3

Capacity = 4

Test Case 2

Number of Items = 4

Weights:

1 2 3 4

Values:

2 4 6 3

Capacity = 6

Sample Output

Test Case 1

Optimal Selection:

Item 0

Item 2

Total Value = 7

Test Case 2

Optimal Selection:

Item 0

Item 1

Item 2

Total Value = 10

Time Complexity

$O(2^n)$

(All possible subsets of items are generated.)

Space Complexity

$O(n)$

Result

Thus, the 0–1 Knapsack Problem was solved using the exhaustive search approach, and the optimal set of items with the maximum total value was obtained successfully.